E23CSEU1873 | LAB9

```
import re
import numpy as np
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential, Model
from tensorflow.keras.layers import (
    Embedding,
    LSTM,
    Dense,
    Dropout,
    Input,
    LayerNormalization,
    MultiHeadAttention,
    GlobalAveragePooling1D,
    Add,
    Conv1D,
)
from tensorflow.keras.callbacks import EarlyStopping

# ========================
# 1) Dataset
# ========================
RAW_TEXT = """
machine learning models learn patterns from data.
sequence models process data step by step.
recurrent neural networks are designed for sequential tasks.
rnn models maintain hidden states across time steps.

long short term memory networks solve long dependency problems.
lstm uses gates to control information flow.
gru models simplify the lstm architecture.
sequence prediction is useful in many applications.

language modeling predicts the next word in a sentence.
speech recognition processes audio sequences.
time series forecasting predicts future values.
music generation creates new melodies.

generative models learn probability distributions.
they generate new samples similar to training data.
sequence generation is widely used in artificial intelligence.
deep learning improves sequence modeling performance.
"""


def clean_text(text: str) -> str:
    """Lowercase and remove extra whitespace; keep only letters and spaces."""
    text = text.lower()
    text = re.sub(r"[^a-z\s]", "", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text


cleaned_text = clean_text(RAW_TEXT)

# Split into sentences for easier sequence generation.
sentences = [s.strip() for s in RAW_TEXT.strip().split("\n") if s.strip()]
sentences = [clean_text(s) for s in sentences]

# ========================
# 2) Tokenization
# ========================
# Use word-level tokenization.
tokenizer = Tokenizer(oov_token="<OOV>")
tokenizer.fit_on_texts(sentences)
word_index = tokenizer.word_index
vocab_size = len(word_index) + 1  # +1 for padding token

print("Vocabulary size:", vocab_size)
print("Sample word index:", dict(list(word_index.items())[:10]))

# Convert all sentences to token sequences.
encoded_sentences = tokenizer.texts_to_sequences(sentences)

# ========================
# 3) Create input-output pairs
# ========================
# For next-word prediction, create sliding windows.
# Example: [machine learning models learn patterns] -> predict 'from'
sequence_length = 5

X, y = [], []
for seq in encoded_sentences:
    if len(seq) <= sequence_length:
        continue
    for i in range(sequence_length, len(seq)):
        X.append(seq[i - sequence_length : i])
        y.append(seq[i])

X = np.array(X)
y = np.array(y)

print("Training samples:", X.shape[0])
print("Input shape:", X.shape)

# ========================
# 4) LSTM-based generative model
# ========================
embedding_dim = 64
lstm_units = 128

lstm_model = Sequential([
    Input(shape=(sequence_length,)),
    Embedding(input_dim=vocab_size, output_dim=embedding_dim),
    LSTM(lstm_units, return_sequences=False),
    Dropout(0.2),
    Dense(128, activation="relu"),
    Dense(vocab_size, activation="softmax"),
])

lstm_model.compile(
    loss="sparse_categorical_crossentropy",
    optimizer="adam",
    metrics=["accuracy"],
)
```

```
lstm_model.summary()

# ========================
# 5) Transformer-based generative model
# ========================

def positional_encoding(max_len: int, d_model: int) -> tf.Tensor:
    """Create sinusoidal positional encoding."""
    positions = np.arange(max_len)[:, np.newaxis]
    dims = np.arange(d_model)[np.newaxis, :]
    angle_rates = 1 / np.power(10000, (2 * (dims // 2)) / np.float32(d_model))
    angle_rads = positions * angle_rates

    pe = np.zeros((max_len, d_model), dtype=np.float32)
    pe[:, 0::2] = np.sin(angle_rads[:, 0::2])
    pe[:, 1::2] = np.cos(angle_rads[:, 1::2])
    return tf.constant(pe[np.newaxis, ...])  # shape: (1, max_len, d_model)


class TransformerBlock(tf.keras.layers.Layer):
    def __init__(self, embed_dim, num_heads, ff_dim, rate=0.1):
        super().__init__()
        self.att = MultiHeadAttention(num_heads=num_heads, key_dim=embed_dim)
        self.ffn = Sequential([
            Dense(ff_dim, activation="relu"),
            Dense(embed_dim),
        ])
        self.layernorm1 = LayerNormalization(epsilon=1e-6)
        self.layernorm2 = LayerNormalization(epsilon=1e-6)
        self.dropout1 = Dropout(rate)
        self.dropout2 = Dropout(rate)

    def call(self, inputs, training=False):
        attn_output = self.att(inputs, inputs)
        attn_output = self.dropout1(attn_output, training=training)
        out1 = self.layernorm1(inputs + attn_output)
        ffn_output = self.ffn(out1)
        ffn_output = self.dropout2(ffn_output, training=training)
        return self.layernorm2(out1 + ffn_output)


def build_transformer_model(vocab_size: int, sequence_length: int, embed_dim=64, num_heads=4, ff_dim=128):
    inputs = Input(shape=(sequence_length,))
    x = Embedding(vocab_size, embed_dim)(inputs)

    # Add positional encoding
    pe = positional_encoding(sequence_length, embed_dim)
    x = x + pe[:, :sequence_length, :]

    x = TransformerBlock(embed_dim, num_heads, ff_dim)(x)
    x = Dropout(0.2)(x)
    x = GlobalAveragePooling1D()(x)
    x = Dense(64, activation="relu")(x)
    outputs = Dense(vocab_size, activation="softmax")(x)

    model = Model(inputs, outputs)
    model.compile(
        loss="sparse_categorical_crossentropy",
        optimizer="adam",
        metrics=["accuracy"],
    )
    return model


transformer_model = build_transformer_model(vocab_size, sequence_length)
transformer_model.summary()

# ========================
# 6) Train models
# ========================
# Because the dataset is tiny, keep epochs moderate.
early_stop = EarlyStopping(monitor="loss", patience=20, restore_best_weights=True)

print("\nTraining LSTM model...")
lstm_model.fit(
    X,
    y,
    epochs=200,
    batch_size=8,
    verbose=0,
    callbacks=[early_stop],
)
print("LSTM training complete.")

print("\nTraining Transformer model...")
transformer_model.fit(
    X,
    y,
    epochs=200,
    batch_size=8,
    verbose=0,
    callbacks=[early_stop],
)
print("Transformer training complete.")

# ========================
# 7) Text generation helpers
# ========================
index_word = {v: k for k, v in word_index.items()}
index_word[0] = ""


def sample_from_probs(probs: np.ndarray, temperature: float = 1.0) -> int:
    """Sample an index from a probability array using temperature."""
    probs = np.asarray(probs).astype("float64")
    probs = np.log(probs + 1e-9) / temperature
    exp_probs = np.exp(probs)
    probs = exp_probs / np.sum(exp_probs)
    return np.random.choice(len(probs), p=probs)


def generate_text(model, seed_text: str, next_words: int = 8, temperature: float = 1.0) -> str:
    seed_text = clean_text(seed_text)
    result = seed_text.split()

    for _ in range(next_words):
        token_list = tokenizer.texts_to_sequences([" ".join(result)])
        token_list = pad_sequences(token_list, maxlen=sequence_length, padding="pre")
        preds = model.predict(token_list, verbose=0)[0]
        next_id = sample_from_probs(preds, temperature=temperature)
        next_word = index_word.get(next_id, "")
        if next_word == "" or next_word == "<OOV>":
            continue
```

```
        result.append(next_word)

    return " ".join(result)


# ========================
# 8) Generate samples
# ========================
seed = "sequence models"

print("\n--- LSTM Generated Samples ---")
for i in range(3):
    print(f"Sample {i+1}: {generate_text(lstm_model, seed, next_words=10, temperature=0.8)}")

print("\n--- Transformer Generated Samples ---")
for i in range(3):
    print(f"Sample {i+1}: {generate_text(transformer_model, seed, next_words=10, temperature=0.8)}")

# ========================
# 9) Optional evaluation helper
# ========================
def perplexity_like(loss_value: float) -> float:
    return float(np.exp(loss_value))

lstm_loss, lstm_acc = lstm_model.evaluate(X, y, verbose=0)
t_loss, t_acc = transformer_model.evaluate(X, y, verbose=0)

print("\n--- Evaluation ---")
print(f"LSTM loss: {lstm_loss:.4f}, accuracy: {lstm_acc:.4f}, perplexity-like: {perplexity_like(lstm_loss):.4f}")
print(f"Transformer loss: {t_loss:.4f}, accuracy: {t_acc:.4f}, perplexity-like: {perplexity_like(t_loss):.4f}")

# Save models if needed
# lstm_model.save('lstm_sequence_model.keras')
# transformer_model.save('transformer_sequence_model.keras')
```

```
Vocabulary size: 88
Sample word index: {'<OOV>': 1, 'models': 2, 'sequence': 3, 'data': 4, 'in': 5, 'learning': 6, 'learn': 7, 'step': 8, 'networks': 9, 'time': 10}
Training samples: 31
Input shape: (31, 5)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 5, 64)	5,632
lstm (LSTM)	(None, 128)	98,816
dropout (Dropout)	(None, 128)	0
dense (Dense)	(None, 128)	16,512
dense_1 (Dense)	(None, 88)	11,352

```
 Total params: 132,312 (516.84 KB)
 Trainable params: 132,312 (516.84 KB)
 Non-trainable params: 0 (0.00 B)
Model: "functional_2"
```

Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 5)	0
embedding_1 (Embedding)	(None, 5, 64)	5,632
add (Add)	(None, 5, 64)	0
transformer_block (TransformerBlock)	(None, 5, 64)	83,200
dropout_4 (Dropout)	(None, 5, 64)	0
global_average_pooling1d (GlobalAveragePooling1D)	(None, 64)	0
dense_4 (Dense)	(None, 64)	4,160
dense_5 (Dense)	(None, 88)	5,720

```
 Total params: 98,712 (385.59 KB)
 Trainable params: 98,712 (385.59 KB)
 Non-trainable params: 0 (0.00 B)

Training LSTM model...
LSTM training complete.

Training Transformer model...
Transformer training complete.

--- LSTM Generated Samples ---
Sample 1: sequence models from data by by step by step information step information
Sample 2: sequence models from data data by step step step step step step
Sample 3: sequence models by from data by step by step step step step

--- Transformer Generated Samples ---
Sample 1: sequence models sequential used forecasting control series are sequential maintain performance tasks
Sample 2: sequence models language recognition by the is generative recognition hidden learning probability
Sample 3: sequence models intelligence time by neural probability long control artificial term samples

--- Evaluation ---
LSTM loss: 0.0011, accuracy: 1.0000, perplexity-like: 1.0011
Transformer loss: 4.5065, accuracy: 0.0000, perplexity-like: 90.6010
```